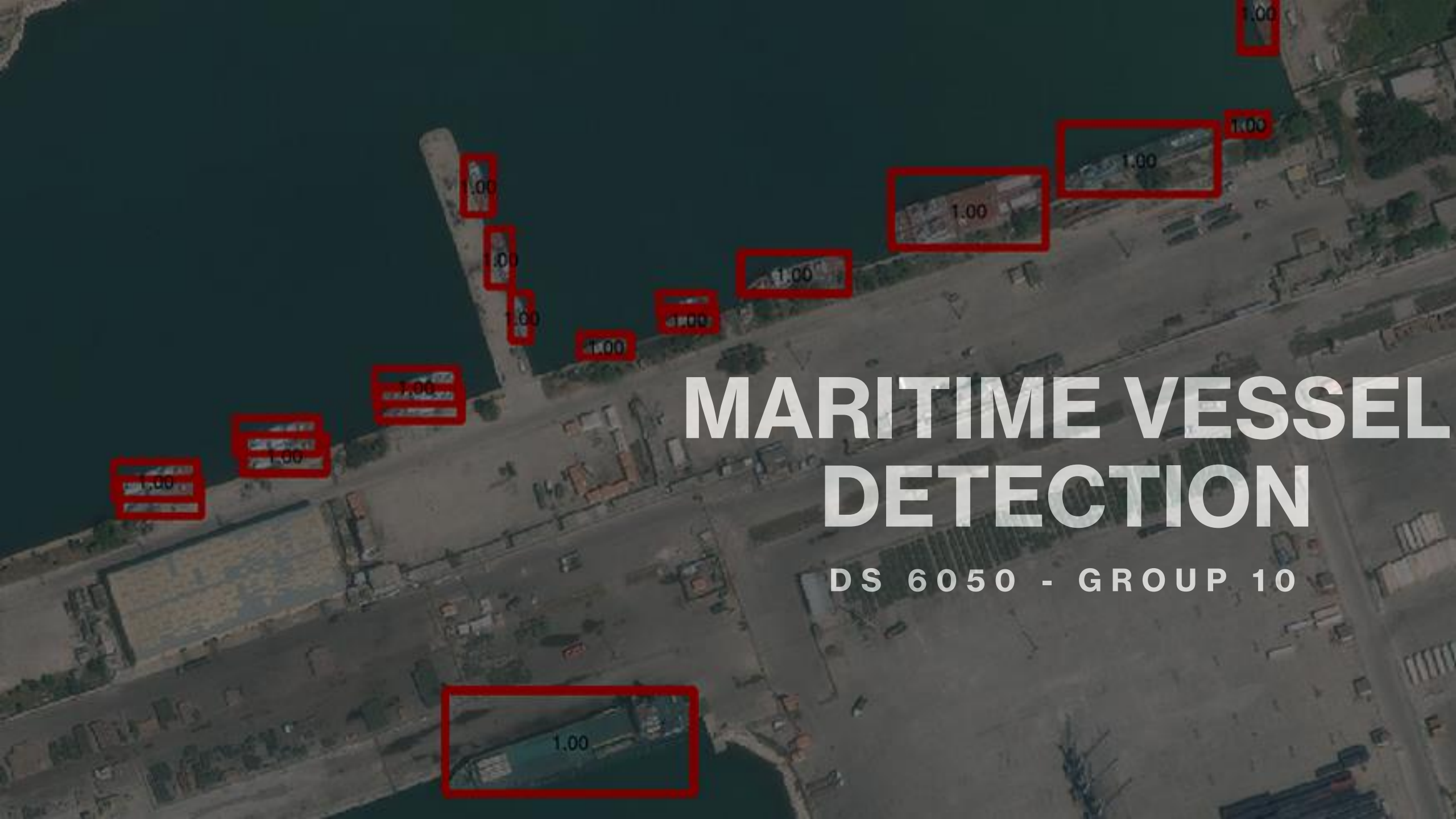


An aerial photograph of a coastal industrial or port area. The image is overlaid with several red rectangular bounding boxes, each containing the number '1.00'. These boxes are positioned along the shoreline, highlighting various structures and areas. The background shows a mix of industrial buildings, parking lots, and some greenery. The text 'MARITIME VESSEL DETECTION' is prominently displayed in the center-right, with 'DS 6050 - GROUP 10' below it.

MARITIME VESSEL DETECTION

DS 6050 - GROUP 10

Group 10

Kevin Guan

Quality Engineer

M.S Data Science (In-Progress)

B.S. Biological Engineering

Bardia Nikpour

DevOps Engineer

M.S Data Science (In-Progress)

B.S Computer Engineering

Zachary Holland

Data Scientist/DevOps Engineer

M.S Data Science (In-Progress)

B.S Computer Science

B.S Applied Mathematics

Problem Statement

This project aims to compare and evaluate several popular deep learning approaches for maritime vessel detection in satellite imagery. We want to:

1. Compare the performance of three widely-used object detection models on the ShipRSImageNet dataset
2. Examine how different backbone architectures affect detection results when used with certain models
3. Analyze the trade-offs between detection accuracy, localization precision, and computational requirements for these different configurations

Motivation & Challenges

Military Applications:

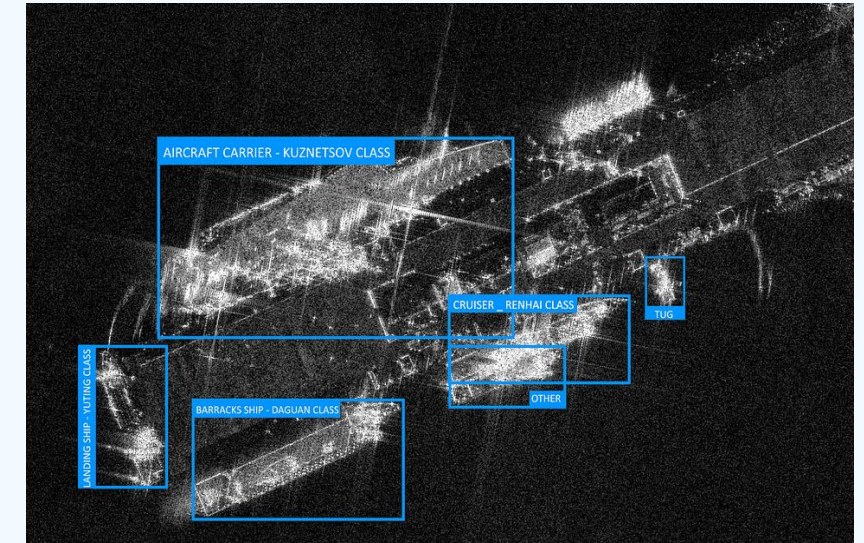
- **Threat Analysis:** Help prevent/detect smuggling, piracy, terrorist activities, military posture
- **Rapid Response:** Faster detection supports data driven decision-making

Commercial Applications:

- **Optimized Shipping Routes:** Improves traffic flow and scheduling
- **Illegal Fishing Detection:** Flags violations for authorities

Challenges

- **High Background Noise:** Water, waves, and clutter cause false positives or hide small vessels
- **Environmental Conditions:** Weather and lighting changes obscure vessels and affect image quality
- **Scalability :** Vast areas and diverse ship types demand efficient, adaptable algorithms



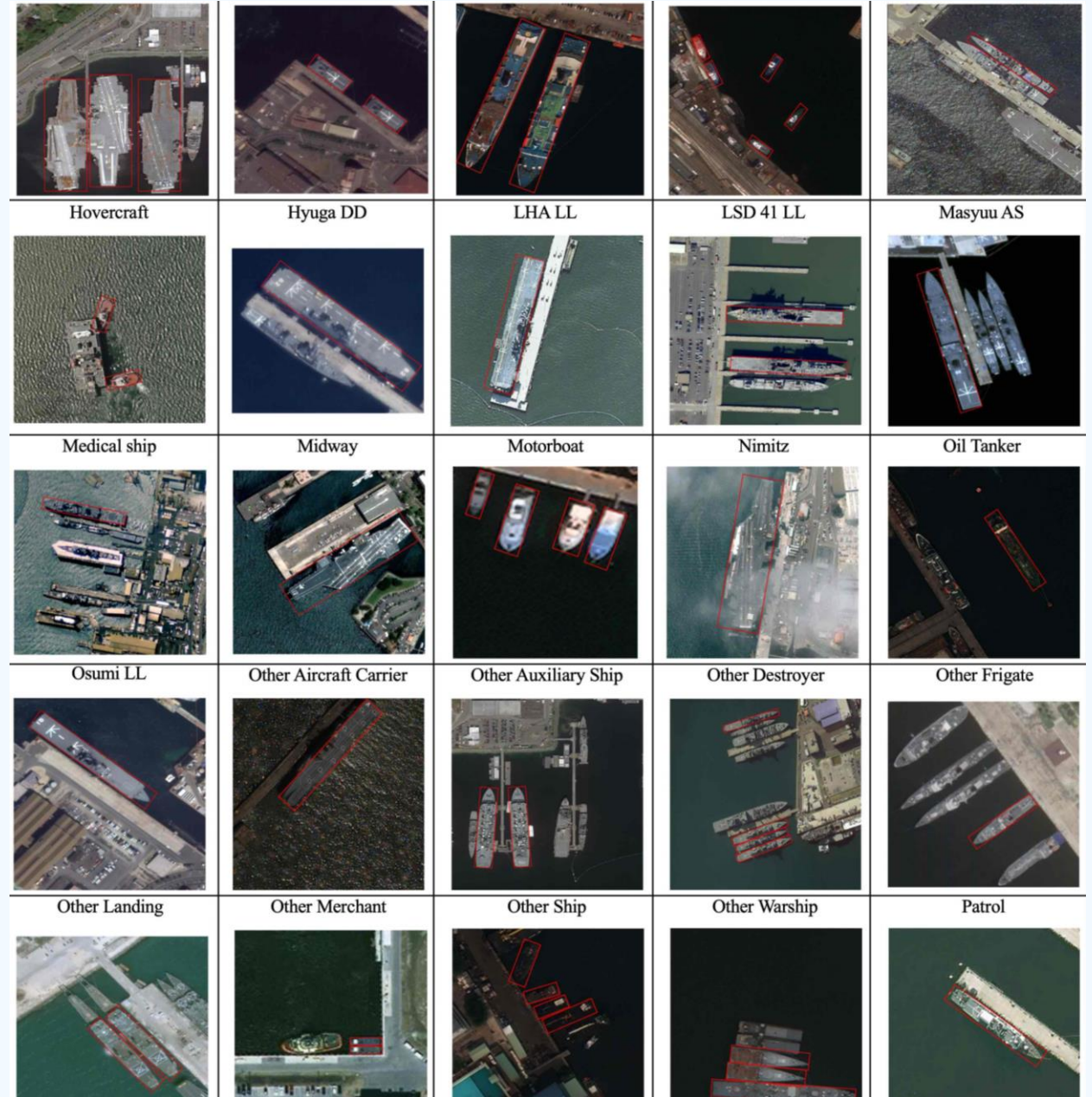
Dataset Overview

Dataset: [ShipRSImageNet](#)

- ~3,435 high-resolution images & 17,573 ship instances
- **Multi-source** data
 - WorldView-3
 - xView
 - HRSC2016
 - FGSD
 - Airbus
 - GaoFen-2 (Chinese Satellite)
 - JiLin-1 (Chinese Satellite)
- 50 ship classes (commercial, military) with horizontal, oriented, and polygon bounding boxes

Characteristics:

- Images around **930×930** pixels
- Highly **variable conditions**:
 - Geography
 - Resolutions
 - Weather



Augmentation & Preprocessing

Annotation Verification

- Inspect and correct bounding boxes or misalignment or label inaccuracies

Image Resizing & Normalization

- Standardize image dimensions and normalize pixel values for consistent model input

Data Splitting

- Divide data into train, validation, and test sets with no scene overlap, ensuring vessel type balance

Augmentation Techniques

- **Geometric Transforms:** Random flips (horizontal/vertical), rotations, cropping
- **Color & Lighting Adjustments:** Brightness/contrast jitter, hue shifts
- **Synthetic Overlays:** Partial cloud cover, wave clutter, fog effects to mimic real-world conditions



Proposed Methodology

Model Comparison:

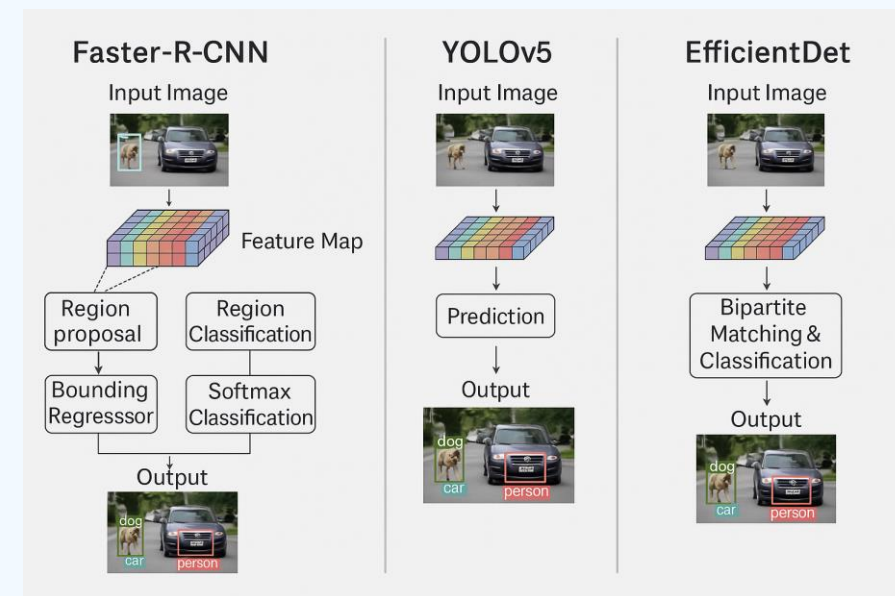
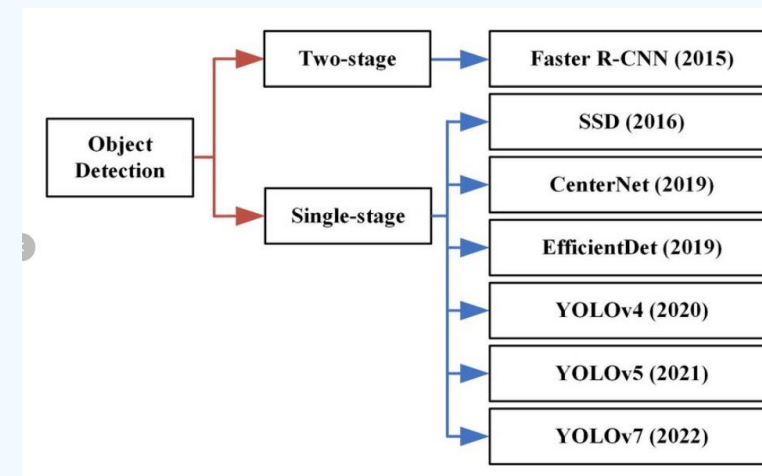
- **Faster R-CNN:** Two-stage detector with a Region Proposal Network (RPN) for high accuracy
- **YOLOv8:** Single-stage, end-to-end detection known for real-time or near real-time performance
- **EfficientDet:** Employs compound scaling for balanced accuracy/efficiency
- **Uniform Training Settings:** Standard preprocessing, data augmentation, same splits for fairness

In-depth R-CNN Backbone Analysis (Faster R-CNN focus):

- **ResNet-50:** Strong general-purpose backbone with pre-integrated FPN; only the classifier head is modified
- **MobileNetV3:** Lightweight and edge-optimized; a solid speed-accuracy trade-off with minimal tuning
- **EfficientNet-B0:** Custom setup with selected layers and FPN, balancing efficiency and accuracy
- **SqueezeNet:** Ultra-compact, using fire modules and a 4-level FPN for constrained environments
- **ShuffleNetV2:** Highly efficient with dynamic FPN logic, ideal for real-time, low-power inference
- **ConvNeXt-Tiny:** Modern ResNet-style backbone with efficient FPN and strong representation quality

Fine-tuning & Optimization Strategies:

- **Regularization:** L1, L2 to handle overfitting
- **Dropout:** Finetune for robust generalization.
- **Learning Rate Schedules:** Step decay, cosine adjustment to stabilize training
- **Data Augmentation:** Reinforce resilience to environmental variability



Results: YOLOv8

YOLOv8-M: Best overall performance

- mAP@0.5: **0.8714**
- Precision: **0.8331**
- FPS: **34.86**

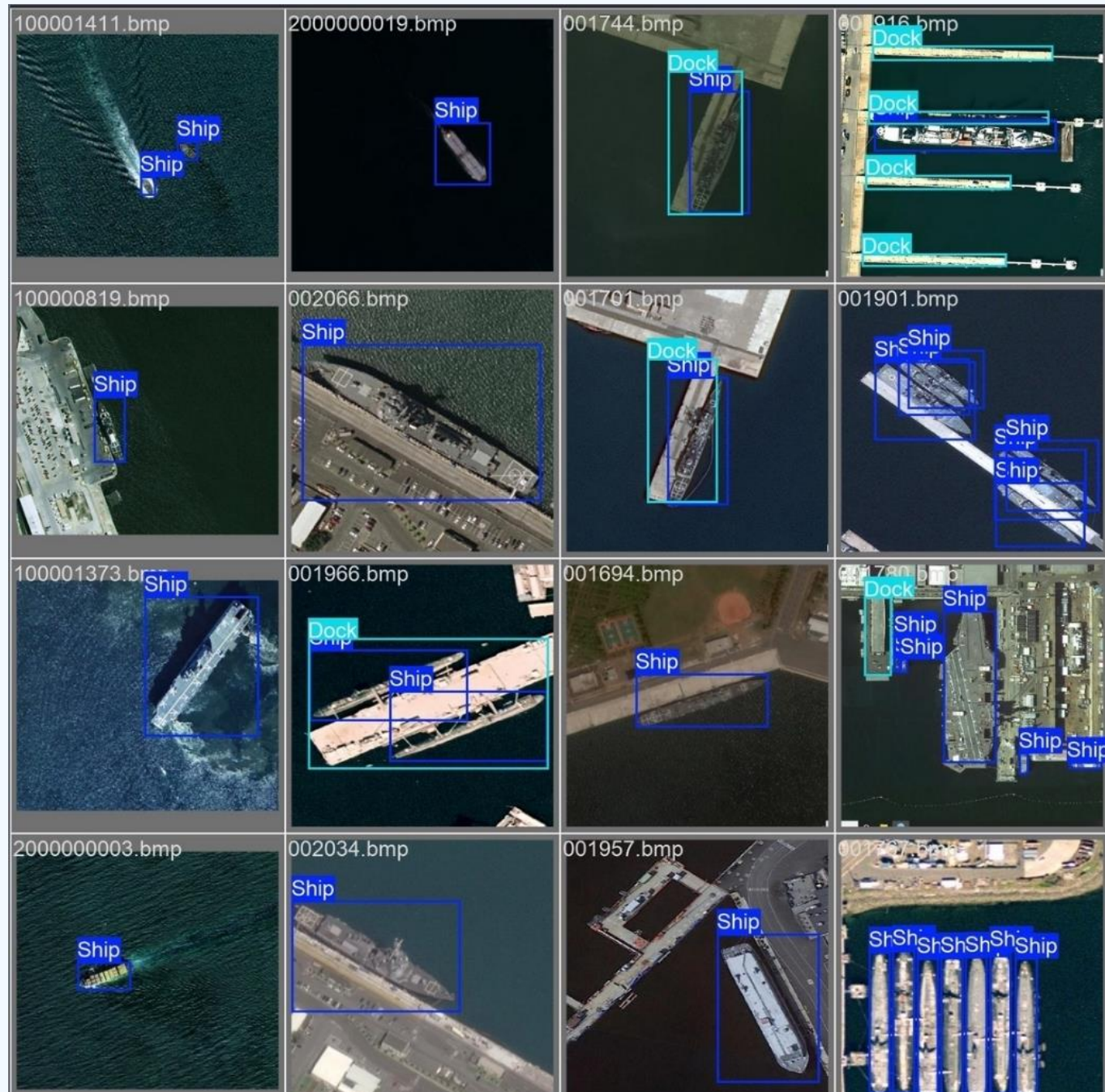
Best balance of accuracy and speed

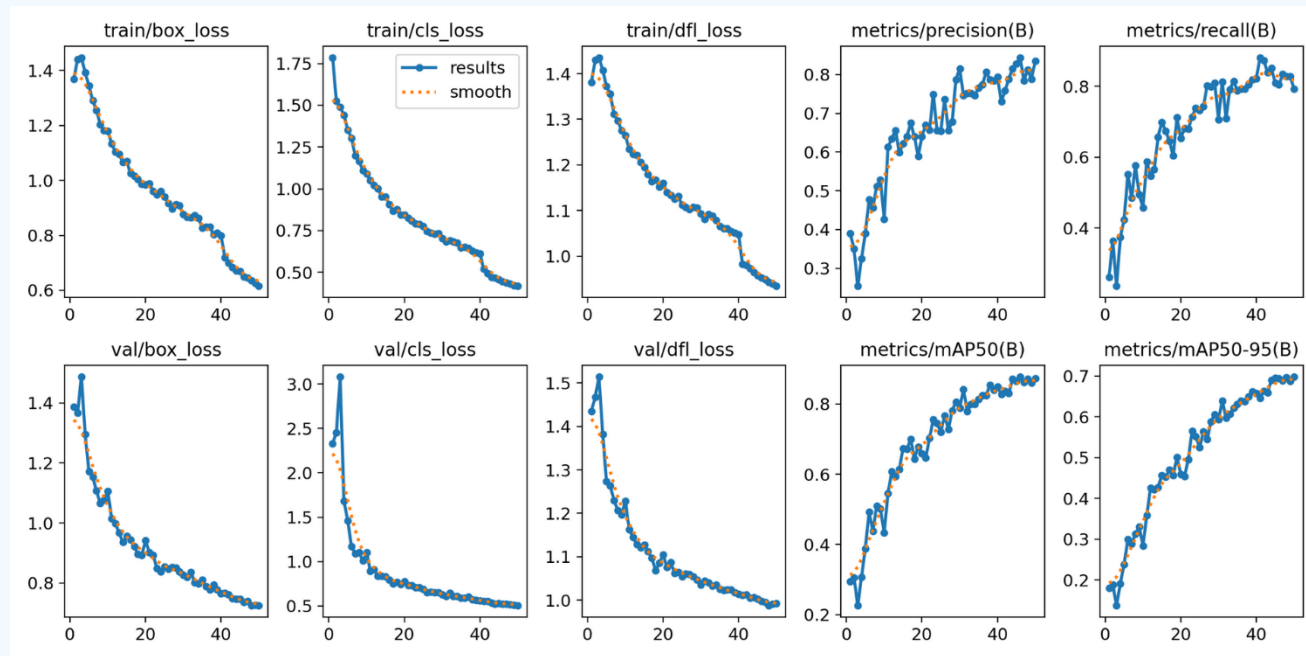
YOLOv8-S: Highest recall (**0.8526**)

- Slightly lower precision and speed
- Good choice when **maximizing detection coverage** is critical

YOLOv8-L: Solid, but underwhelming

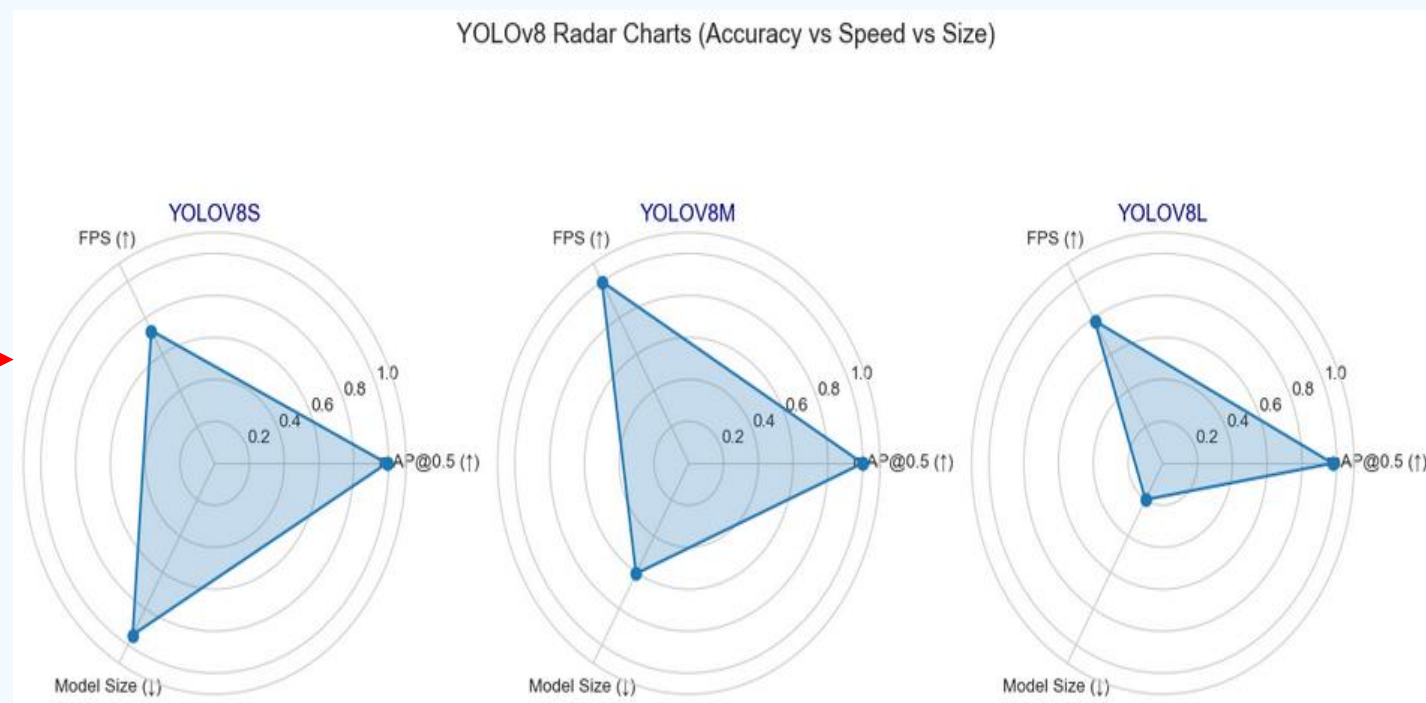
- No accuracy gain over M
- Slower inference (27.34 FPS) → *less efficient for real-time use*





Training Results

Radar chart highlighting performance



Results: EfficientDet

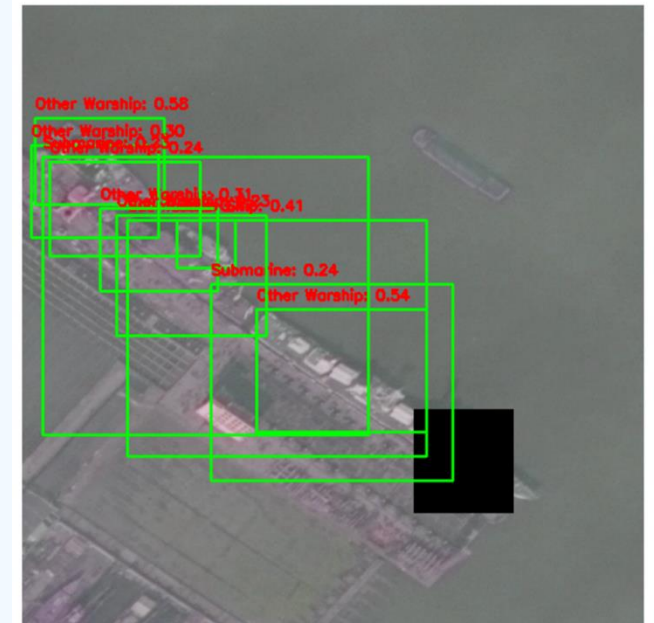
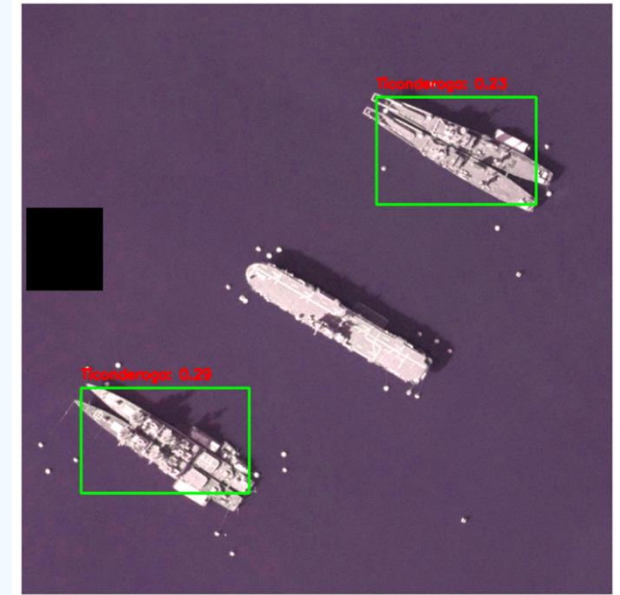
EfficientDet – Final Metrics

- **Classification Loss:** 0.159
- **Regression Loss:** 0.179
- **Batch Loss:** 0.338
- **Total Loss:** 0.353
- **Training Duration:** 20 epochs

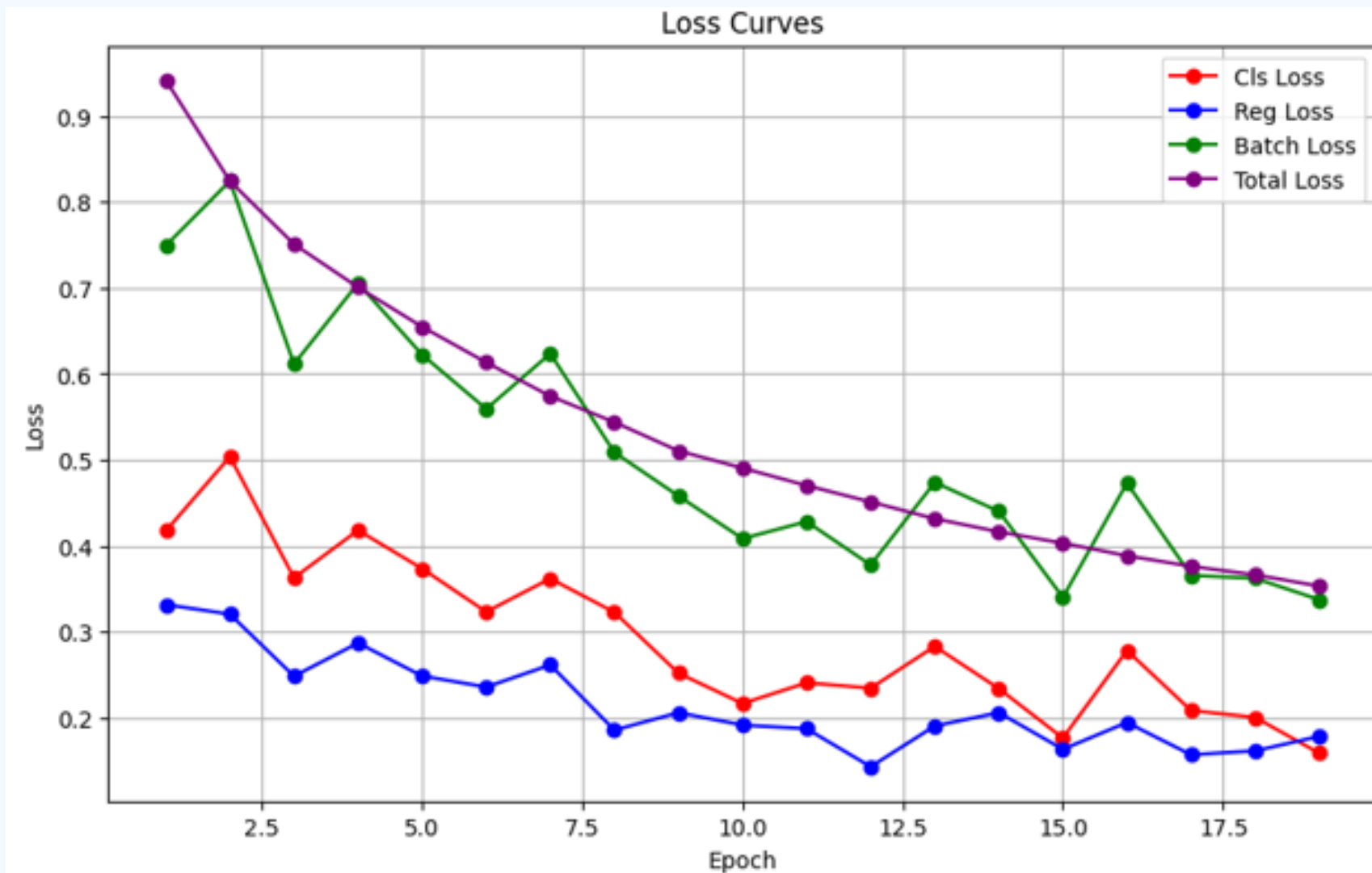
Data Tripling via augmentation

Improved performance vs. 10-epoch baseline (Total Loss: 0.681 → 0.353)

Only model set up to classify the ***type of vessel***, not just location



EfficientDet



Results: Faster R-CNN

ResNet-50 (mAP: 0.4738)

→ Best overall **detection accuracy**, but was slow and resource-heavy

MobileNet-V3 (mAP: 0.3662)

→ Best **speed-accuracy trade-off**, suitable for real-time applications with limited compute

EfficientNet-B0 (mAP: 0.2123)

→ Moderate recall but poor precision; a **middle-ground** option with limited utility

ShuffleNet-V2 (mAP: 0.0448)

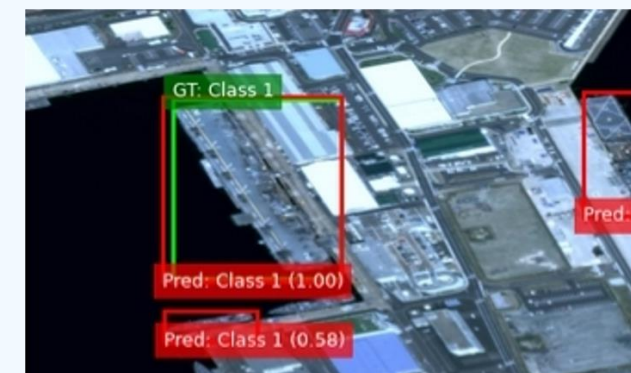
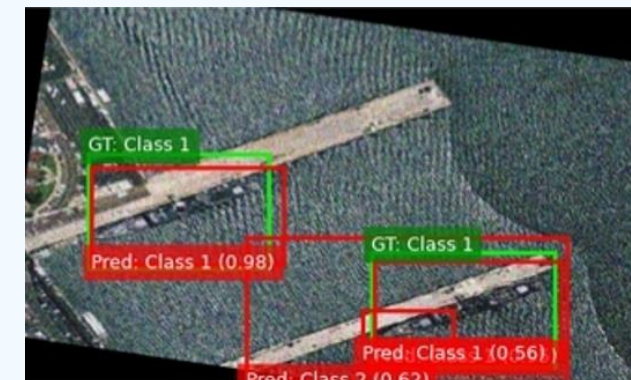
→ Fast but extremely low accuracy – **not suitable** for high-resolution vessel detection.

SqueezeNet (mAP: 0.1714)

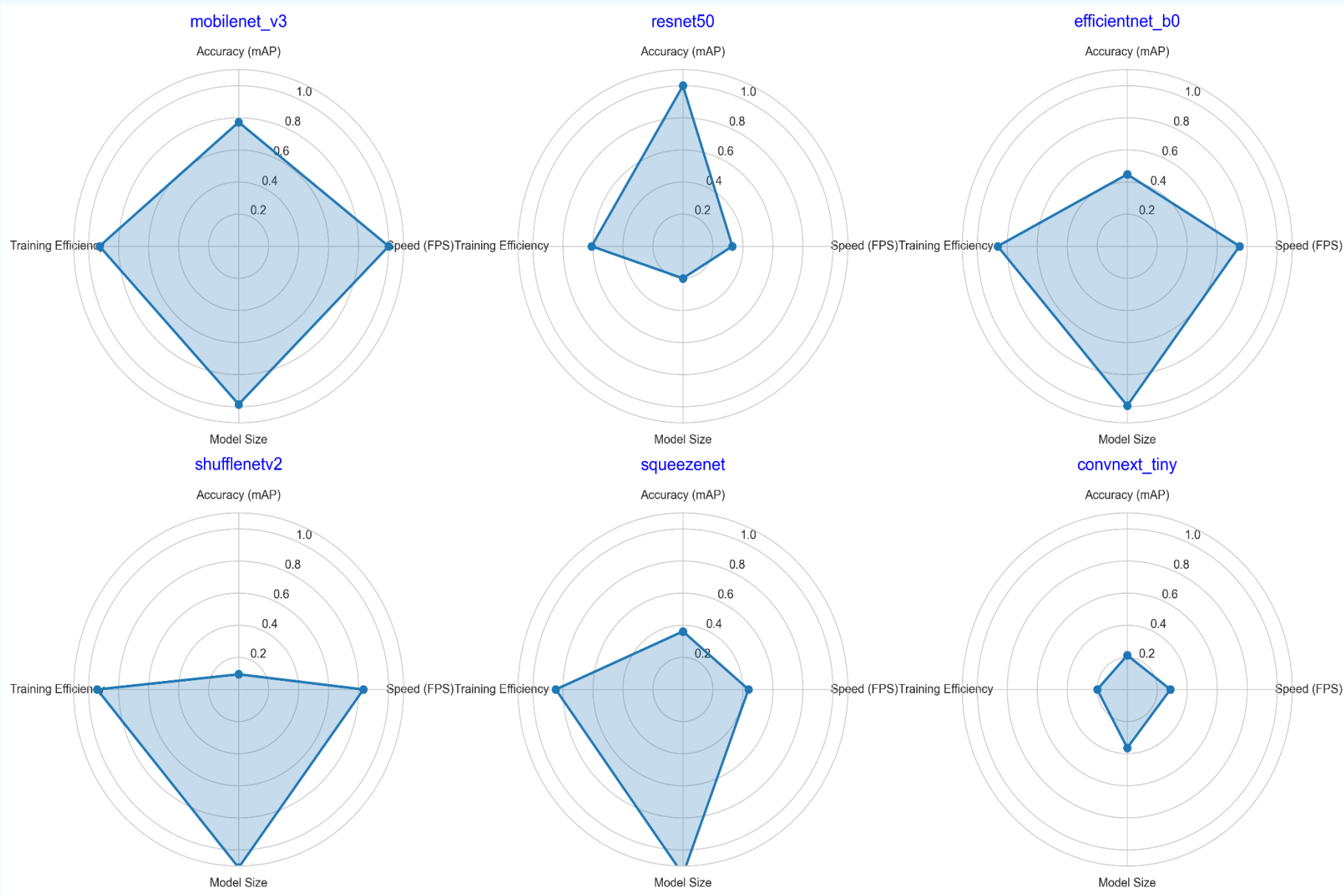
→ Small and deployable, but **too weak** for meaningful results.

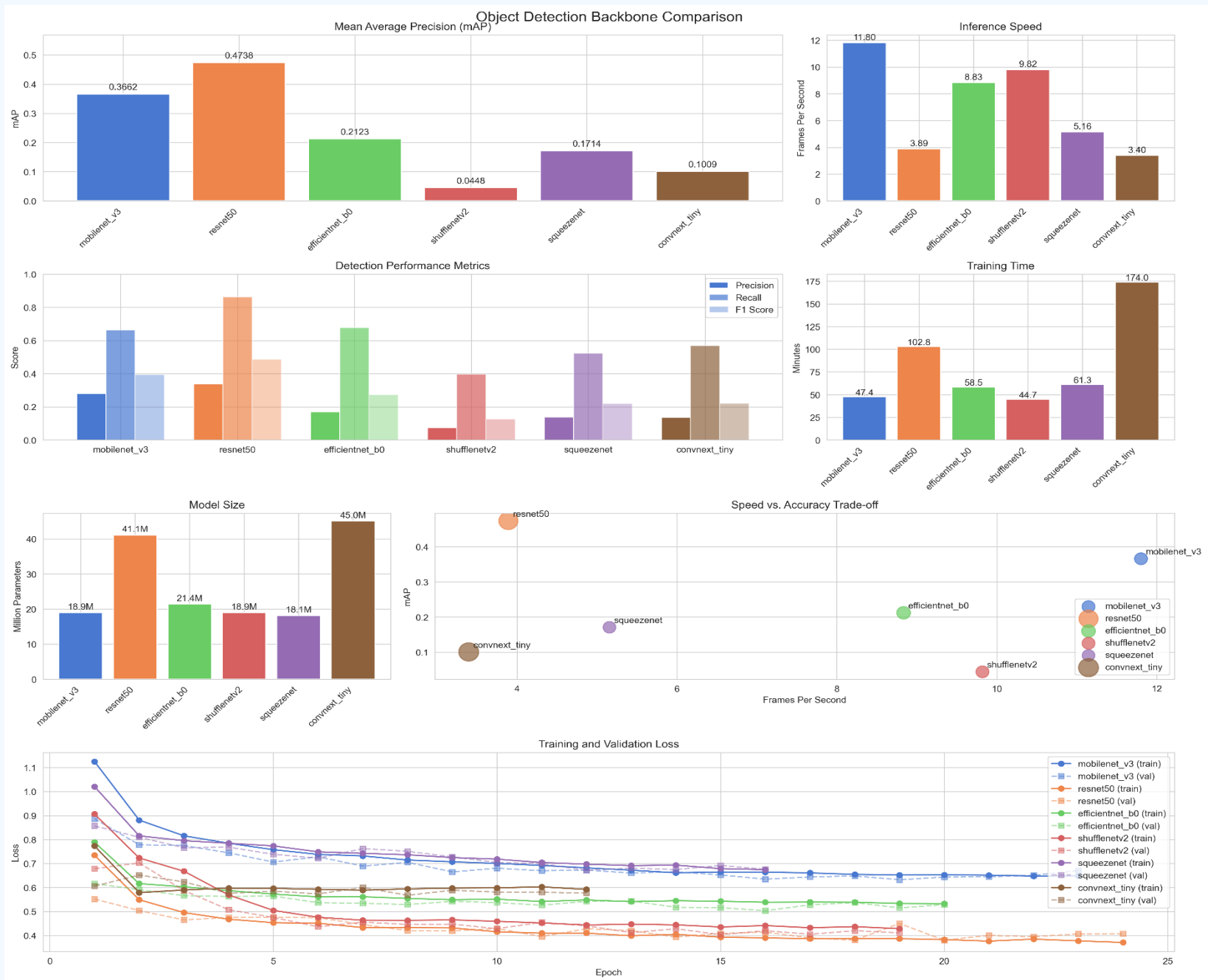
ConvNeXt-Tiny (mAP: 0.1009)

→ Slowest and underperformed on all fronts – **not viable** for continued use.



Faster R-CNN





Interesting Finds



Cloud Coverage

In this image, dense cloud cover makes it difficult to detect the ship, highlighting how environmental conditions can obscure targets and reduce model reliability.



The Sandbar Problem

We have a very complex scene here. The model correctly identified the target but misclassified a sandbar as a vessel with 87% confidence, revealing the difficulty of distinguishing similar-looking structures.

Conclusion

Research Questions:

1. Best backbone for Faster R-CNN for accurate vessel localization?

→ *Faster R-CNN + ResNet-50*, though slow, had the highest localization precision

2. Impact of backbone networks?

→ Backbones significantly affected both speed and accuracy. ResNet-50 maximized precision, MobileNetV3 offered strong balance, and ShuffleNetV2/SqueezeNet were too weak for complex scenes

3. Best model trade-off?

→ *YOLOv8-M* was absolutely the most balanced model, combining high accuracy (mAP@0.5 of 0.8714) with great inference, making it ideal for practical deployment for this use case

Thanks for listening!